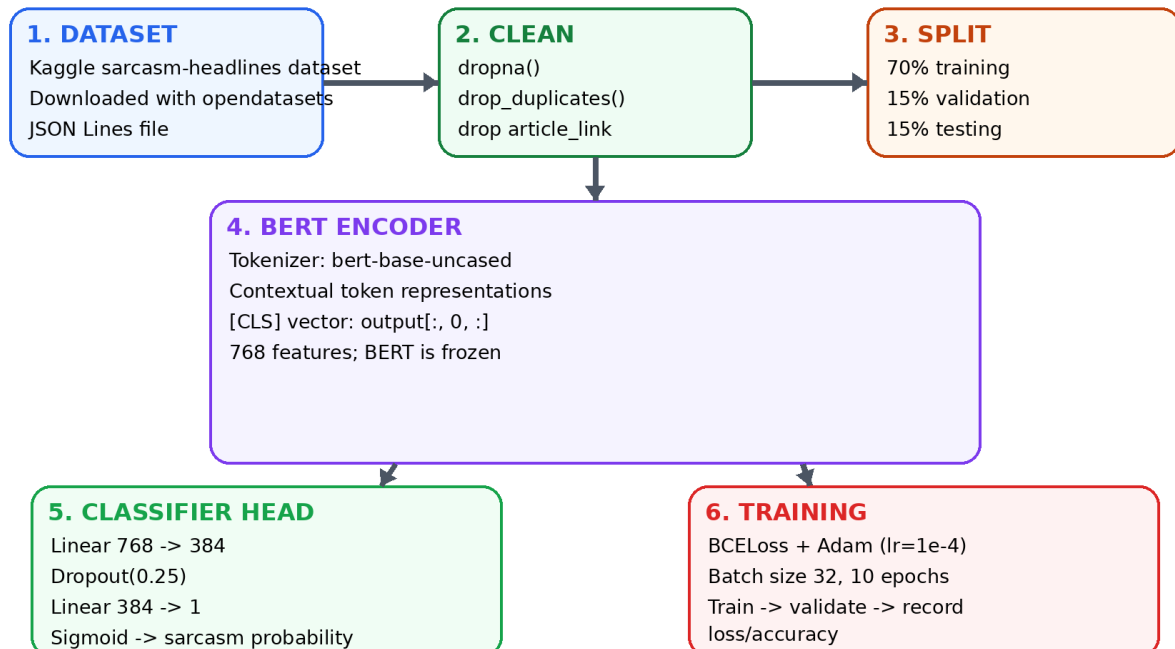


BERT-Based Sarcasm Detection

Code explanation for `text_classification_DL05.ipynb`

BERT Sarcasm Detection - Code Architecture

Notebook pipeline based on `text_classification_DL05.ipynb`



Final evaluation loop exists in the notebook.

Important: the printed "test accuracy" uses `total_acc_train`, so it is training accuracy, not test accuracy.

The notebook also divides accumulated loss by 1000 for plotting: this is a display choice, not the usual mean-batch loss.

What this project does: it classifies news headlines as sarcastic or non-sarcastic using a pretrained BERT encoder and a custom PyTorch binary-classification head.

1. Dataset download and setup

The first notebook cell installs **transformers** and **opendatasets**, then downloads the Kaggle sarcasm dataset. The code also includes a compatibility workaround for Python 3.13 because the notebook observed that opendatasets may attempt to import the removed `cgi` module.

```
od.download("https://www.kaggle.com/datasets/rmisra/news-headlines-dataset-for-sarcasm-detection")
```

2. Libraries and device selection

The notebook imports PyTorch, Hugging Face Transformers, scikit-learn utilities, Pandas, NumPy, and Matplotlib. It selects CUDA when available and otherwise falls back to CPU.

```
device = "cuda" if torch.cuda.is_available() else "cpu"
```

The recorded notebook output shows **cuda** was available during the saved run.

3. Loading and cleaning the dataset

The dataset is read as JSON Lines using Pandas. Missing rows and duplicates are removed, and `article_link` is discarded because the model only needs the headline text and the sarcasm label.

```
data_df = pd.read_json(..., lines=True)
data_df.dropna(inplace=True)
data_df.drop_duplicates(inplace=True)
data_df.drop(["article_link"], inplace=True, axis=1)
```

The saved run reports **26,708 rows and 2 columns**: headline and is_sarcastic.

4. Train / validation / test split

The notebook first reserves 30% of the data, then splits that portion equally into validation and test sets, producing approximately **70% training, 15% validation, and 15% testing**.

Subset	Rows	Approx. share
Training	18,695	70%
Validation	4,006	15%
Testing	4,007	15%

5. BERT tokenizer and encoder

The notebook loads google-bert/bert-base-uncased. The tokenizer converts each headline into token IDs and an attention mask. Each example is limited to 100 tokens and padded/truncated to that fixed length.

```
tokenizer(x, max_length=100, truncation=True, padding="max_length", return_tensors="pt")
```

The BERT encoder generates contextual representations for all tokens. The custom model then takes the first token representation, output[:, 0, :], corresponding to the classifier-style **[CLS]** position in this implementation. That vector has 768 features.

6. Dataset class and DataLoader

A custom PyTorch Dataset stores tokenized headlines and labels. Three dataset objects are created for training, validation, and testing. A DataLoader batches those examples with a batch size of 32.

7. Classification head

The model wraps BERT inside a custom PyTorch module. The classification head is intentionally small:

```
Linear(768, 384) -> Dropout(0.25) -> Linear(384, 1) -> Sigmoid
```

The sigmoid converts the final logit into a probability-like value between 0 and 1. The notebook then uses rounding at 0.5 to convert that value into a binary class during accuracy calculation.

8. Transfer learning: freezing BERT

Before creating the final model, every BERT parameter is frozen with `requires_grad = False`. This means the run trains only the added classifier layers rather than fine-tuning the full BERT encoder. That reduces the number of trainable parameters and makes the experiment lighter than full BERT fine-tuning.

9. Loss and optimizer

The notebook uses binary cross-entropy loss (BCELoss) because the target is binary and the model explicitly outputs a sigmoid probability. Adam is used with a learning rate of **1e-4**.

```
criterion = nn.BCELoss()
optimizer = Adam(model.parameters(), lr=LR)
```

10. Training loop - what happens in each epoch

For each training batch, the notebook performs the standard PyTorch sequence: get inputs and labels, run a forward pass, compute loss, calculate gradients with `backward()`, update weights with the optimizer, and clear gradients with `zero_grad()`.

```
prediction = model(input_ids, attention_mask)
batch_loss = criterion(prediction, label)
batch_loss.backward()
optimizer.step()
optimizer.zero_grad()
```

Accuracy is calculated by comparing rounded sigmoid predictions with the labels. After training, the model switches to evaluation mode and validates inside `torch.no_grad()`, so validation does not build a gradient graph.

11. Training results stored in the notebook

Epoch	Train acc.	Val. acc.
1	82.04%	85.80%
2	83.02%	86.45%
3	82.97%	86.25%
4	83.40%	87.14%
5	83.66%	87.19%
6	83.89%	86.35%
7	83.54%	87.34%
8	83.51%	86.84%
9	83.56%	86.74%
10	83.83%	87.47%

The best recorded validation accuracy is **87.4688%** at epoch 10.

12. Test-evaluation section and important bug

The notebook does run a test loop and accumulates `total_acc_test`. However, the final print statement accidentally uses the training accumulator:

```
print(f"Accuracy score on testing Data is :{... total_acc_train ...}")
```

Therefore the displayed **83.8299%** is the training accuracy, not the test accuracy. The correct reporting expression should use `total_acc_test` divided by the length of the testing dataset. This distinction matters when presenting results in a portfolio or interview.

13. Loss / accuracy visualization

The final cell plots training vs. validation loss and training vs. validation accuracy for the 10 epochs. These curves are useful for spotting convergence and possible overfitting, although the notebook currently scales the accumulated loss by dividing by 1000 rather than computing a conventional average across batches.

14. Practical ML-engineering lessons

Concept	Where it appears
Data ingestion	Kaggle download + Pandas <code>read_json</code>
Data cleaning	<code>dropna</code> , duplicate removal, column selection
Dataset splitting	train / validation / test

Representation	BERT contextual embeddings
Transfer learning	Frozen pretrained BERT
Deep learning	PyTorch model + backpropagation
Experiment tracking	Loss and accuracy lists per epoch
Evaluation discipline	Separate validation and test loops
Debugging mindset	Detecting incorrect test-score reporting

15. Suggested next improvements

For a stronger GitHub portfolio version, fix the test-metric bug, compute mean loss, add precision/recall/F1 and a confusion matrix, set a random seed for reproducibility, save the trained classifier, and create a small inference script that accepts a headline and returns a prediction. These improvements extend the existing notebook without changing its core idea.

Source basis: this document explains the code and recorded run outputs contained in the uploaded notebook; no outside results are substituted for the notebook results.